

Tecnología Digital VI

Ensamblés II – Camino e Interpretabilidad

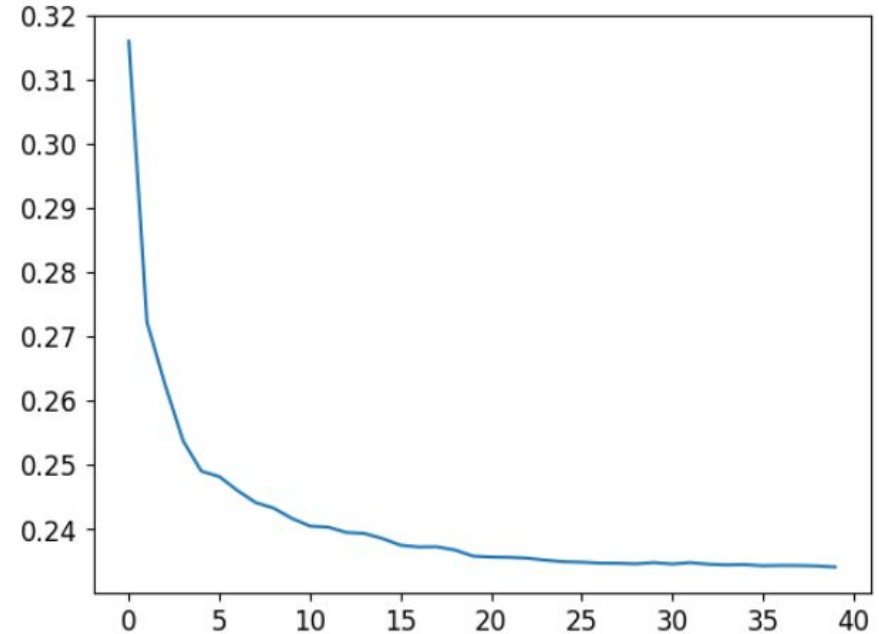
Licenciatura en Tecnología Digital – UTDT

Random Forest – Cantidad de árboles

En primer lugar, podemos ver cómo afecta la cantidad de árboles en el bosque a nuestra métrica en el conjunto de validación.

Esto nos ayuda a entender cómo mejora el modelo pero no qué está haciendo.

```
preds = np.stack([t.predict(valid_xs) for t in m.estimators_])  
plt.plot([r_mse(preds[:i+1].mean(0), valid_y) for i in range(40)]);
```



Random Forest – Feature Importance

El modelo Random Forest de Scikit-learn ya trae incorporado un atributo feature_importances_ que nos da un valor numérico de qué tan importante es una columna.

Ahora podemos empezar a visualizar qué está teniendo en cuenta el modelo para hacer las predicciones.

```
def rf_feat_importance(m, df):  
    return pd.DataFrame({'cols':df.columns, 'imp':m.feature_importances_  
        }).sort_values('imp', ascending=False)
```

```
fi = rf_feat_importance(m, xs)  
fi[:10]
```

	cols	imp
58	YearMade	0.186833
6	ProductSize	0.127272
30	Coupler_System	0.075358
7	fiProductClassDesc	0.070345
32	Hydraulics_Flow	0.059941
66	saleElapsed	0.049191
55	ModelID	0.048763
3	fiSecondaryDesc	0.045518
31	Grouser_Tracks	0.037843
12	Enclosure	0.036103

Tiene sentido pensar que el año en que se fabricó una máquina y su tamaño va a influir en el precio

Random Forest – Cálculo de feature importance

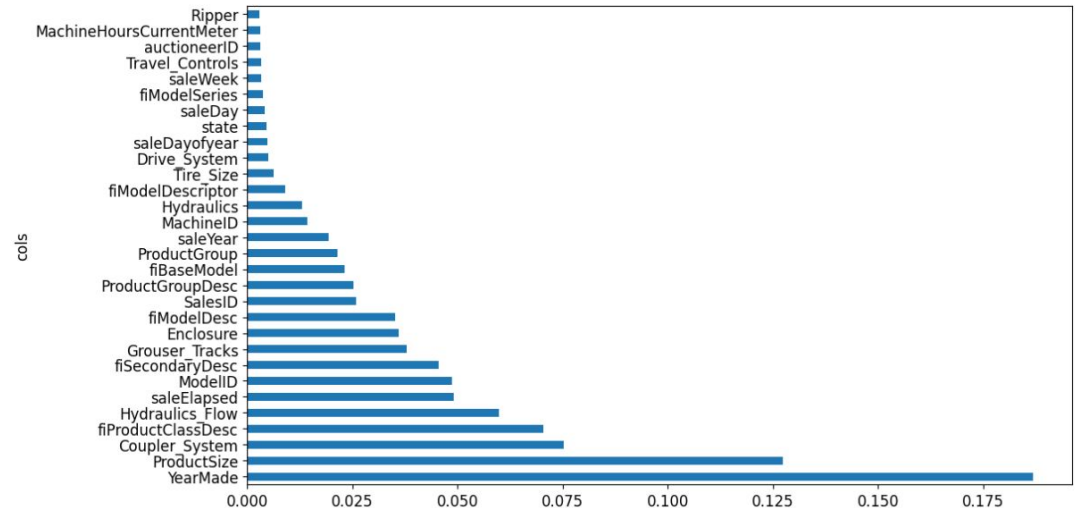
The way these importances are calculated is quite simple yet elegant. The feature importance algorithm loops through each tree, and then recursively explores each branch. At each branch, it looks to see what feature was used for that split, and how much the model improves as a result of that split. The improvement (weighted by the number of rows in that group) is added to the importance score for that feature. This is summed across all branches of all trees, and finally the scores are normalized such that they add to 1.

Otra manera de calcular la feature importance: una vez entrenado nuestro modelo, tomamos la columna a la cual queremos calcularle su importancia, la shuffleamos y volvemos a utilizar el modelo de predicción. El deterioro en la medida de predicción se puede utilizar como medida de importancia del feature.

Random Forest – Feature Importance

Ahora tenemos un gráfico de feature importance que no tiene *mucho* que envidiarle en cuanto a interpretabilidad a un modelo lineal.

```
def plot_fi(fi):  
    return fi.plot('cols', 'imp', 'barh', figsize=(12,7), legend=False)  
  
plot_fi(fi[:30]);
```



Random Forest – Feature Selection

Esta información se puede utilizar para hacer Feature Selection.

Al hacerlo no solo conseguimos un modelo más simple sino que además está funcionando mejor sobre el conjunto de validación.

```
to_keep = fi[fi.imp>0.005].cols  
len(to_keep)
```

21

```
xs_imp = xs[to_keep]  
valid_xs_imp = valid_xs[to_keep]
```

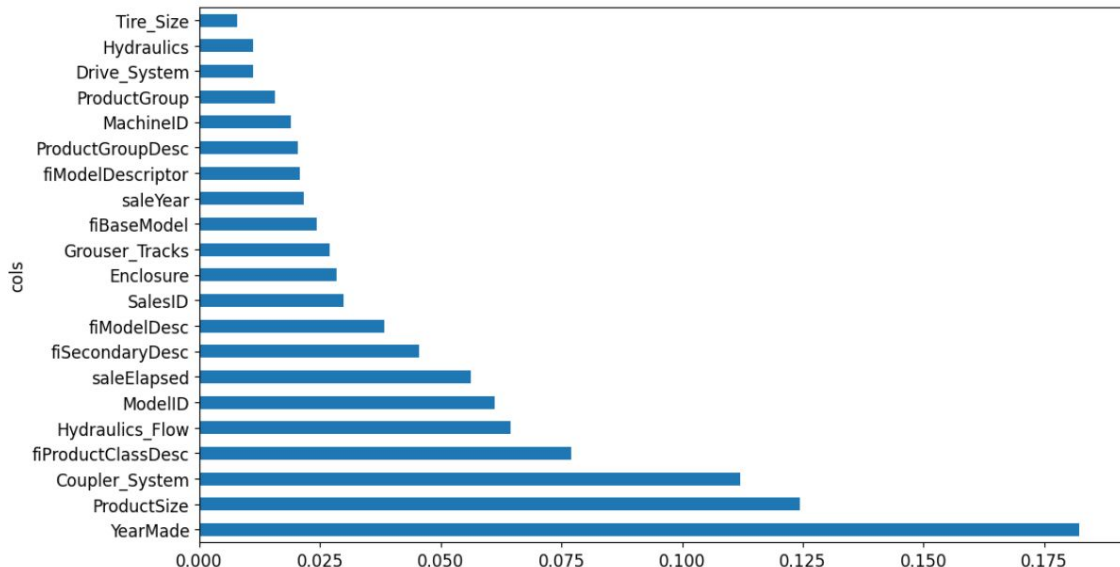
```
m = rf(xs_imp, y)
```

```
m_rmse(m, xs_imp, y), m_rmse(m, valid_xs_imp, valid_y)  
(0.181015, 0.231821)
```

Random Forest – Feature Selection

Ahora con muchas menos columnas podemos tener un gráfico más limpio y claro.

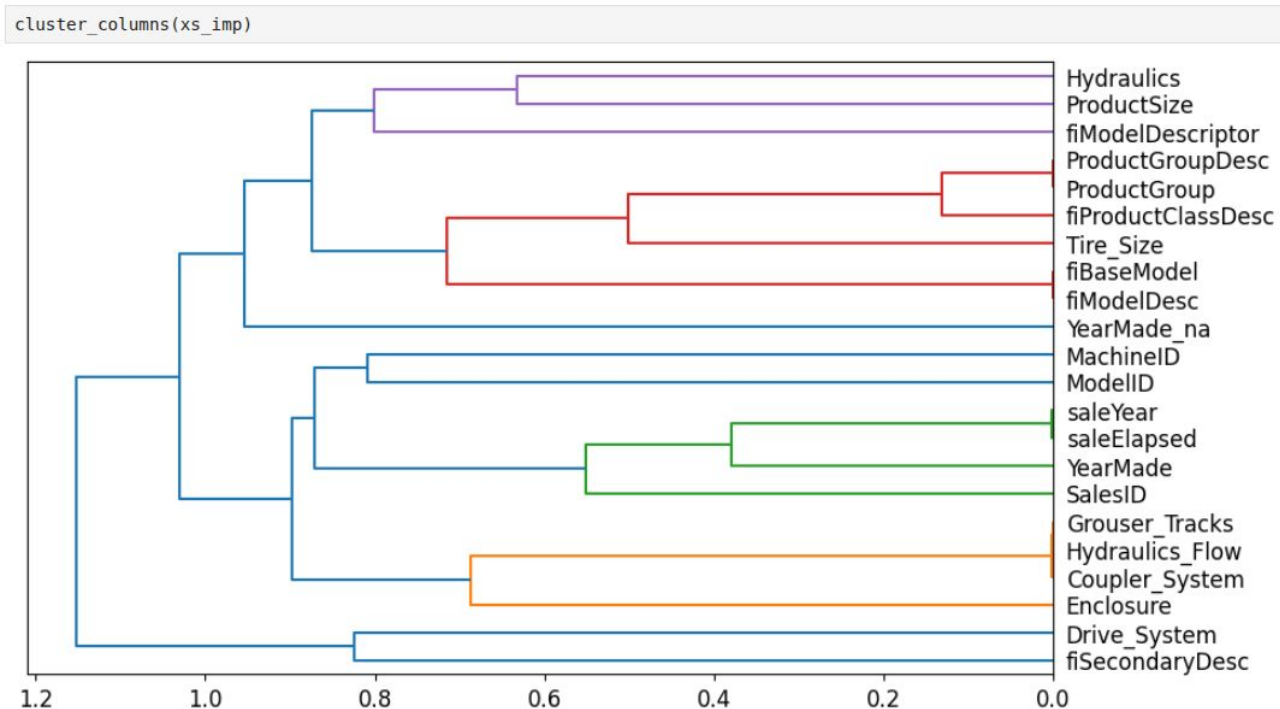
```
plot_fi(rf_feat_importance(m, xs_imp));
```



Random Forest – Interpretación

Otra forma posible de interpretación (y de Feature Selection) es entender qué columnas son *parecidas* para el modelo.

Para esto podemos hacer un clustering jerárquico aglomerativo (¿un qué?) que vaya agrupando las columnas.



Más a la derecha = más

Random Forest – Cluster Columns

En general, cuando clusterizamos, lo hacemos en función de una medida de distancia de puntos en el espacio.

Este caso es parecido pero en vez de tomar puntos y una distancia euclídea en el espacio, los “puntos” a clusterizar serán las columnas o features del dataset y en vez de distancia euclídea se utiliza la medida de rank correlation.

El rank correlation entre dos columnas lo que mide es la correlación que tienen los puntos en cada columna según su orden dentro de esa columna, sin importar el valor absoluto (dado que a los árboles de decisión el valor absoluto no es algo que les importe).

Random Forest – Feature Selection

El clustering sirve como interpretación pero también como material para hacer Feature Selection. Podemos ver qué sucede al sacar un representante de cada cluster y así decidir qué features descartar.

```
def get_oob(df):  
    m = RandomForestRegressor(n_estimators=40, min_samples_leaf=15,  
                             max_samples=50000, max_features=0.5, n_jobs=-1, oob_score=True)  
    m.fit(df, y)  
    return m.oob_score_
```

```
get_oob(xs_imp)
```

```
0.8778402552203507
```

```
{c:get_oob(xs_imp.drop(c, axis=1)) for c in (  
    'saleYear', 'saleElapsed', 'ProductGroupDesc', 'ProductGroup',  
    'fiModelDesc', 'fiBaseModel',  
    'Hydraulics_Flow', 'Grouser_Tracks', 'Coupler_System')}
```

```
{'saleYear': 0.8775896228512217,  
 'saleElapsed': 0.8732593254588217,  
 'ProductGroupDesc': 0.878550063354707,  
 'ProductGroup': 0.8781817887980742,  
 'fiModelDesc': 0.8765091824171777,  
 'fiBaseModel': 0.8768243055074915,  
 'Hydraulics_Flow': 0.8779969217835307,  
 'Grouser_Tracks': 0.878239377304804,  
 'Coupler_System': 0.8779898290795644}
```

Random Forest – Partial Dependence

Otro análisis de interpretación del modelo que se puede realizar es el de Partial Dependence.

La idea en este análisis es observar cómo depende el valor a predecir de una columna en particular.

Para hacer esto podríamos tomar la columna a analizar y por cada valor de la columna simplemente tomar el promedio de la variable target. Esto no es incorrecto pero podemos hacer algo un poco mejor.

Vamos a hacerlo con las columnas *ProductSize* y *YearMade* que fueron las dos de mayor feature importance.

Random Forest – Partial Dependence

Dijimos que no nos conformábamos con tomar el promedio de la variable target para cada valor del feature a analizar. Entonces, ¿cómo se calcula esta *partial dependence*?

Lo que hacemos es, una vez entrenado nuestro modelo, reemplazar la columna a analizar por cada uno de sus valores posibles y calcular el promedio de las predicciones. Es decir, por cada valor que puede tomar la columna, reemplazamos toda esa columna por dicho valor en cada fila, utilizamos el modelo para predecir cada nueva fila y luego tomamos el promedio de la predicción.

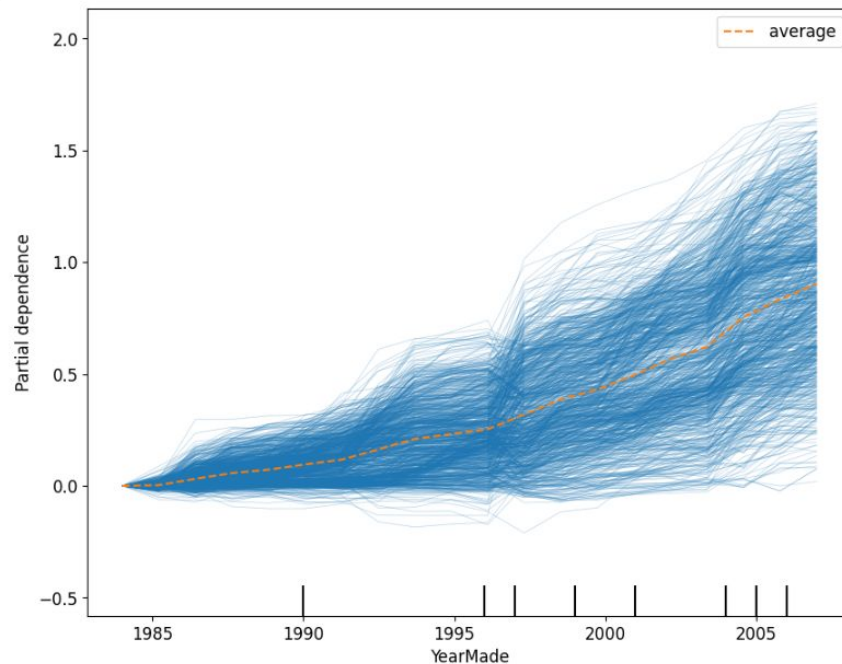
Random Forest – ICE plot

El Partial Dependence Plot toma el promedio para cada valor posible de la columna a analizar.

El Individual Conditional Expectation plot (ICE) grafica la evolución de todas las filas de nuestro dataset.

```
fig, ax = plt.subplots(figsize=(10, 8))

common_params = {"grid_resolution": 20, "random_state": 0}
features_info = {"features": ['YearMade'], "kind": "both", "centered": True}
display = PartialDependenceDisplay.from_estimator(
    m,
    valid_xs_final,
    **features_info,
    ax=ax,
    **common_params,
)
```



Random Forest- Contributions

Otro análisis posible es el de contribuciones. En este análisis podemos ver para cada fila como se llega al valor final predicho.

Para esto se toma el valor promedio de la columna a predecir y luego se van mostrando las contribuciones que tiene cada columna para llegar al valor final de la fila.

```
from treeinterpreter import treeinterpreter
from waterfall_chart import plot as waterfall
```

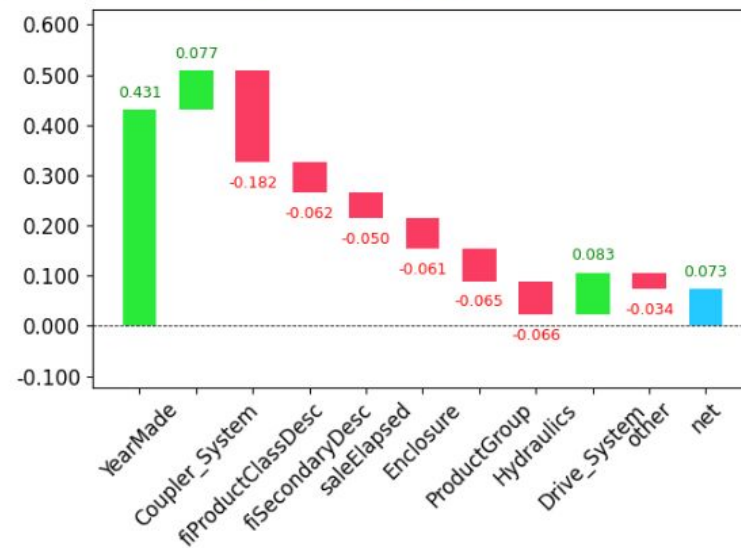
```
row = valid_xs_final.iloc[:5]
```

```
prediction, bias, contributions = treeinterpreter.predict(m, row.values)
```

```
prediction[4], bias[4], contributions[4].sum()
```

```
(array([10.17705413]), 10.10434023018098, 0.07271390343623342)
```

```
waterfall(valid_xs_final.columns, contributions[4], threshold=0.08,
          rotation_value=45, formatting='{:,.3f}');
```



Random Forest – Resumen

Es un **ensamble muy poderoso** para usar en la práctica en problemas tabulares o estructurados.

Son ***bastante inmunes*** al overfitting.

No necesitan un gran preprocesamiento. **No necesitan escalar features o regularizaciones particulares.**

No todo es felicidad: no tienen **ninguna capacidad de extrapolación.**

Random Forest – Extrapolación

Los árboles de decisión **siempre están tomando promedios de muestras que ven** en el conjunto de entrenamiento.

Si una **nueva muestra** está **muy lejos** de lo que vieron, no tienen nada para hacer al respecto.

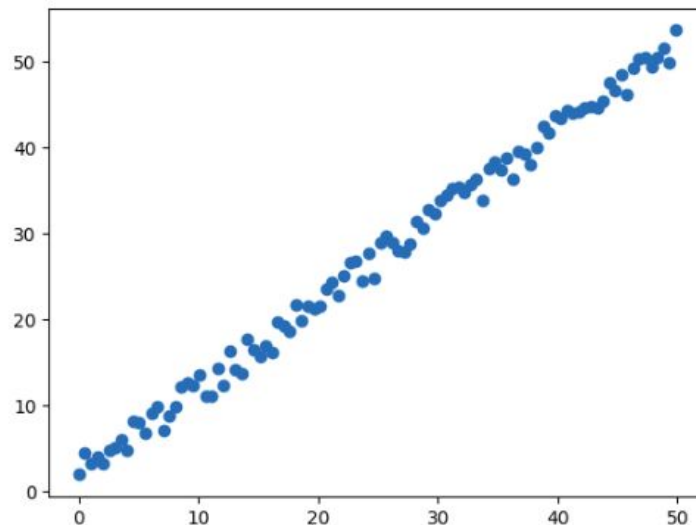
Veamos un ejemplo concreto con una recta creciente.

¿Qué va a predecir un random forest con valores de x mayores a 50?

Va a predecir como si $x = 50$.

```
import numpy as np
import random
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestRegressor
```

```
x = np.linspace(0,75,150)
x_train = x[:100]
y = np.linspace(0,50,100)
y = x_train + 4*np.random.rand(100)
plt.scatter(x_train, y);
```



Random Forest – Extrapolación

Un **RandomForest** no puede hacer ningún tipo de extrapolación.

En este caso caerá por las hojas del árbol que lo lleven más a la derecha del eje x y replicará todas las veces ese valor.

De todas maneras, hay formas de tratar evitar estos problemas.

```
rf = RandomForestRegressor()  
rf.fit(x_train.reshape(-1,1), y)  
predictions = rf.predict(x.reshape(-1,1))  
plt.scatter(x_train, y);  
plt.scatter(x, predictions);
```

